

Atividade 1

Teoria dos Conjuntos

1. Identificação do aluno

Aluno: Sinezio Junio Porfirio Araujo

Matrícula: 2212130034

Curso: Ciência da Computação

Turno: Matutino

Disciplina: Estruturas Matemáticas para Computação

2. Descrição do problema

A atividade consiste em desenvolver um programa em Python que receba dois conjuntos finitos A e B e aplique os conceitos de Teoria dos Conjuntos estudados na disciplina. A implementação utiliza números inteiros informados pelo usuário, separados por espaços, e elimina valores repetidos na formação dos conjuntos.

O programa deve calcular união, interseção, diferenças $A - B$ e $B - A$, cardinalidades de A, B, da união e da interseção, conjuntos das partes e suas cardinalidades. Deve também apresentar uma partição de A, calcular o produto cartesiano $A \times B$ e verificar as inclusões $A \subseteq B$ e $B \subseteq A$.

Os resultados são apresentados no terminal, com identificação do aluno e títulos para cada etapa. O objetivo é relacionar cada definição matemática a uma sequência explícita de instruções, utilizando recursos básicos da linguagem.

3. Procedimento matemático

3.1 União, interseção e diferenças

União: $A \cup B = \{x \mid x \in A \text{ ou } x \in B\}$. Reúne os elementos dos dois conjuntos, sem repetições.

Interseção: $A \cap B = \{x \mid x \in A \text{ e } x \in B\}$. Reúne apenas os elementos comuns.

Diferenças: $A - B = \{x \in A \mid x \notin B\}$ e $B - A = \{x \in B \mid x \notin A\}$. A ordem dos conjuntos influencia o resultado.

3.2 Cardinalidade

A cardinalidade $|A|$ corresponde ao número de elementos distintos de A. A contagem também é aplicada a B, $A \cup B$ e $A \cap B$. Uma relação de conferência é:

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

3. Procedimento matemático (continuação)

3.3 Conjunto das partes

O conjunto das partes $P(A)$ é formado por todos os subconjuntos de A , incluindo o conjunto vazio e o próprio A . Cada elemento pode ser incluído ou não em um subconjunto; portanto, $|P(A)| = 2^{|A|}$. A mesma relação vale para B .

Para construir $P(A)$, inicia-se com $\{\emptyset\}$. A cada elemento de A , são criadas cópias dos subconjuntos existentes com esse elemento acrescentado. Os novos subconjuntos são então reunidos aos anteriores. Por exemplo, se $A = \{1, 2\}$, então $P(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ e $|P(A)| = 4$.

3.4 Partição

Uma partição de A é uma família de subconjuntos não vazios, dois a dois disjuntos, cuja união é A . Neste programa, A é dividido entre números pares e ímpares. Os grupos vazios são descartados. Para $A = \{1, 2, 3, 4\}$, obtém-se $\{\{2, 4\}, \{1, 3\}\}$. Todo inteiro pertence a exatamente um desses grupos.

3.5 Produto cartesiano e inclusão

O produto cartesiano é $A \times B = \{(a, b) \mid a \in A \text{ e } b \in B\}$. Cada elemento de A é combinado com todos os elementos de B , gerando pares ordenados. Sua cardinalidade é $|A \times B| = |A| \cdot |B|$.

A inclusão $A \subseteq B$ ocorre quando todo elemento de A pertence a B . Para verificá-la, procura-se um elemento de A que não pertença a B ; se ele existir, a inclusão é falsa. A verificação de $B \subseteq A$ é feita de modo análogo. Se ambas forem verdadeiras, os conjuntos são iguais.

4. Explicação da implementação

O arquivo **conjuntos.py** solicita nome, matrícula e turno com `input()` e imprime esses dados junto ao nome da disciplina. Em seguida, recebe os elementos de A e B . O método `split()` separa a entrada, `int()` converte cada valor em inteiro e a condição `not in` evita duplicatas nas listas.

As listas representam conjuntos sem repetições e preservam a ordem de entrada. Os comandos `for`, `if`, `in`, `not in` e `append()` implementam a união, a interseção e as diferenças. As cardinalidades são calculadas com contadores incrementados em laços, sem utilizar uma função pronta para obter o resultado matemático.

Os conjuntos das partes são listas de listas, iniciadas por `[[[]]]`. Uma lista temporária armazena os novos subconjuntos antes de inseri-los no resultado. A partição usa o resto da divisão por 2; o produto cartesiano usa dois laços aninhados e tuplas; as inclusões usam variáveis booleanas inicialmente verdadeiras.

A saída utiliza colchetes por empregar listas: `[]` representa o conjunto vazio e `[[[]]]` representa seu conjunto das partes. A ordem de exibição não altera o significado matemático dos conjuntos. Não são necessárias bibliotecas adicionais.

5. Exemplos de testes e resultados

Os testes foram executados no arquivo conjuntos.py e seus resultados foram comparados com as definições matemáticas. Todos os sete casos apresentados foram aprovados. Nas tabelas, os resultados são escritos em notação de conjuntos; no terminal, o programa utiliza listas.

5.1 Casos previstos no enunciado

Caso e entradas	União, interseção e diferenças
T1 - Elementos em comum A = {1, 2, 3, 4} B = {3, 4, 5, 6}	$A \cup B = \{1, 2, 3, 4, 5, 6\}$ $A \cap B = \{3, 4\}$ $A - B = \{1, 2\}; B - A = \{5, 6\}$
T2 - Conjuntos disjuntos A = {1, 2, 3} B = {4, 5, 6}	$A \cup B = \{1, 2, 3, 4, 5, 6\}$ $A \cap B = \emptyset$ $A - B = \{1, 2, 3\}; B - A = \{4, 5, 6\}$
T3 - Inclusão própria A = {1, 2} B = {1, 2, 3, 4}	$A \cup B = \{1, 2, 3, 4\}$ $A \cap B = \{1, 2\}$ $A - B = \emptyset; B - A = \{3, 4\}$
T4 - Conjuntos iguais A = {1, 2, 3} B = {1, 2, 3}	$A \cup B = A \cap B = \{1, 2, 3\}$ $A - B = \emptyset; B - A = \emptyset$

Caso	A ; B	A ∪ B ; A ∩ B	P(A) ; P(B)	A ⊆ B; B ⊆ A
T1	4; 4	6; 2	16; 16	Não; Não
T2	3; 3	6; 0	8; 8	Não; Não
T3	2; 4	4; 2	4; 16	Sim; Não
T4	3; 3	3; 3	8; 8	Sim; Sim

No teste T1, a relação de cardinalidade resulta em $6 = 4 + 4 - 2$. No teste T2, a interseção vazia confirma que os conjuntos são disjuntos. Em T3, somente $A \subseteq B$ é verdadeira. Em T4, as duas inclusões são verdadeiras, confirmando a igualdade.

5.2 Exemplo completo com conjuntos pequenos

T5: entradas A: **1 2** e B: **2 3**. Os resultados obtidos foram:

$A \cup B = \{1, 2, 3\}; A \cap B = \{2\}; A - B = \{1\}; B - A = \{3\}$.

$|A| = 2; |B| = 2; |A \cup B| = 3; |A \cap B| = 1$.

$P(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}; |P(A)| = 4$.

$P(B) = \{\emptyset, \{2\}, \{3\}, \{2, 3\}\}; |P(B)| = 4$.

Partição de A = $\{\{2\}, \{1\}\}$.

$A \times B = \{(1, 2), (1, 3), (2, 2), (2, 3)\}$.

$A \subseteq B$: não; $B \subseteq A$: não.

5. Exemplos de testes (continuação)

5.3 Casos complementares

T6 - Entradas repetidas: A: **2 2 4 4** e B: **4 6 6**. O programa formou $A = \{2, 4\}$ e $B = \{4, 6\}$. Foram obtidos $A \cup B = \{2, 4, 6\}$, $A \cap B = \{4\}$, $A - B = \{2\}$ e $B - A = \{6\}$. As cardinalidades de A e B foram 2, e cada conjunto das partes apresentou 4 subconjuntos. A partição de A foi $\{\{2, 4\}\}$, sem incluir um bloco vazio para os ímpares. As duas inclusões foram falsas.

T7 - Conjunto vazio: a entrada de A foi deixada em branco e B recebeu **1 2**. O resultado foi $A = \emptyset$, $A \cup B = \{1, 2\}$, $A \cap B = \emptyset$, $A - B = \emptyset$ e $B - A = \{1, 2\}$. Foram obtidos $|A| = 0$, $|B| = 2$, $|A \cup B| = 2$ e $|A \cap B| = 0$. O programa apresentou $P(A) = \{\emptyset\}$, com cardinalidade 1, e $P(B) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$, com cardinalidade 4.

Nesse último caso, o produto cartesiano foi vazio, $A \subseteq B$ foi verdadeira e $B \subseteq A$ foi falsa. A partição retornada foi a família vazia, representada por [], compatível com a convenção de partição do conjunto vazio.

6. Análise dos resultados

Os resultados obtidos nos sete testes foram consistentes com as operações solicitadas. A remoção de duplicatas preservou a definição de conjunto; as diferenças respeitaram a ordem dos operandos; e as cardinalidades dos conjuntos das partes corresponderam a 2 elevado ao número de elementos de cada conjunto.

Também foram conferidos os subconjuntos gerados, os pares do produto cartesiano e as duas inclusões em cada teste. Na partição, verificou-se que os blocos eram não vazios, disjuntos e que sua união recuperava A. O caso vazio foi tratado conforme a convenção indicada no teste T7.

A implementação é adequada ao estudo de conjuntos finitos pequenos. A geração do conjunto das partes cresce exponencialmente: um conjunto com n elementos possui 2^n subconjuntos. Por isso, entradas muito extensas aumentam o tempo, a memória utilizada e o volume de saída.

A entrada deve conter apenas números inteiros separados por espaços. A versão analisada não trata textos ou números decimais inválidos; nesses casos, a conversão por `int()` pode interromper a execução. Dentro do domínio testado, o programa realizou as nove tarefas previstas na atividade.

Materiais utilizados

- **Portfólio de Programação Matemática - Parte 1.** Documento oficial da disciplina, seções 7, 9 e 10.
- **Guia de Dúvidas e Orientações - Portfólio de Programação Matemática.** Material de apoio, seções 10, 11, 14, 17 e 18.
- **conjuntos.py.** Código da Atividade 1, localizado em `P1/atividade_01_conjuntos/`, utilizado nas execuções de teste.